MODULE 5 {Spring core and MAVEN}

Exercise 1: Configuring a Basic Spring Application

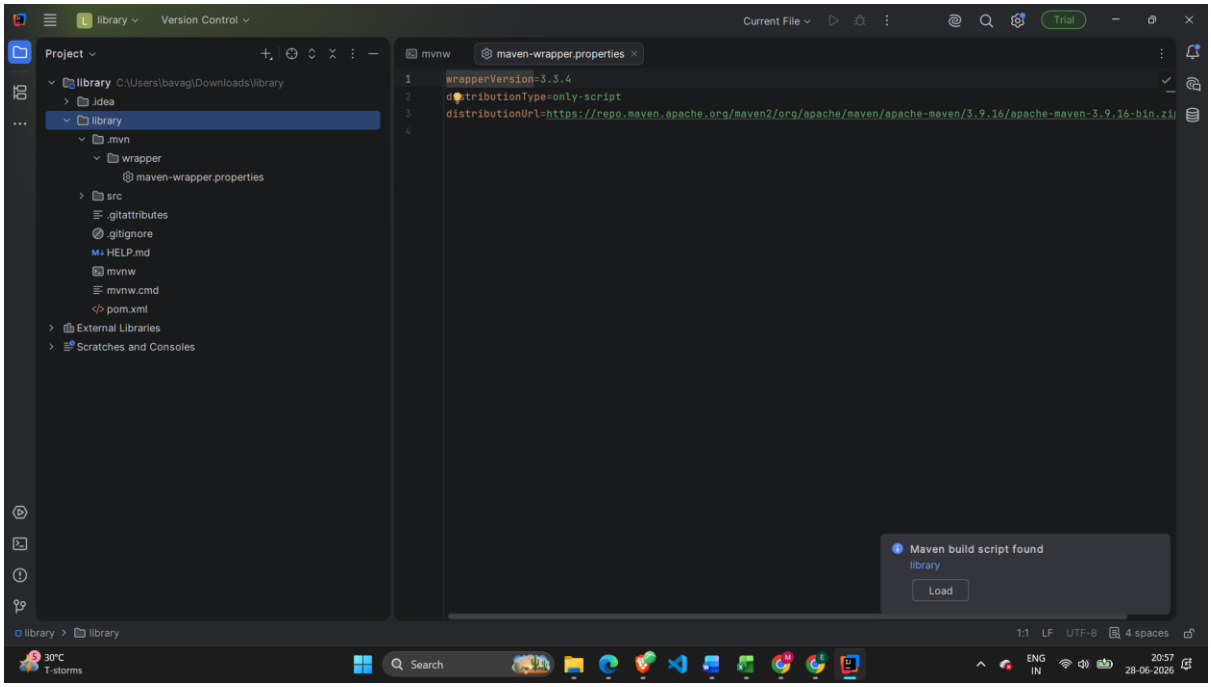
Scenario:

Your company is developing a web application for managing a library. You need to use the Spring Framework to handle the backend operations.

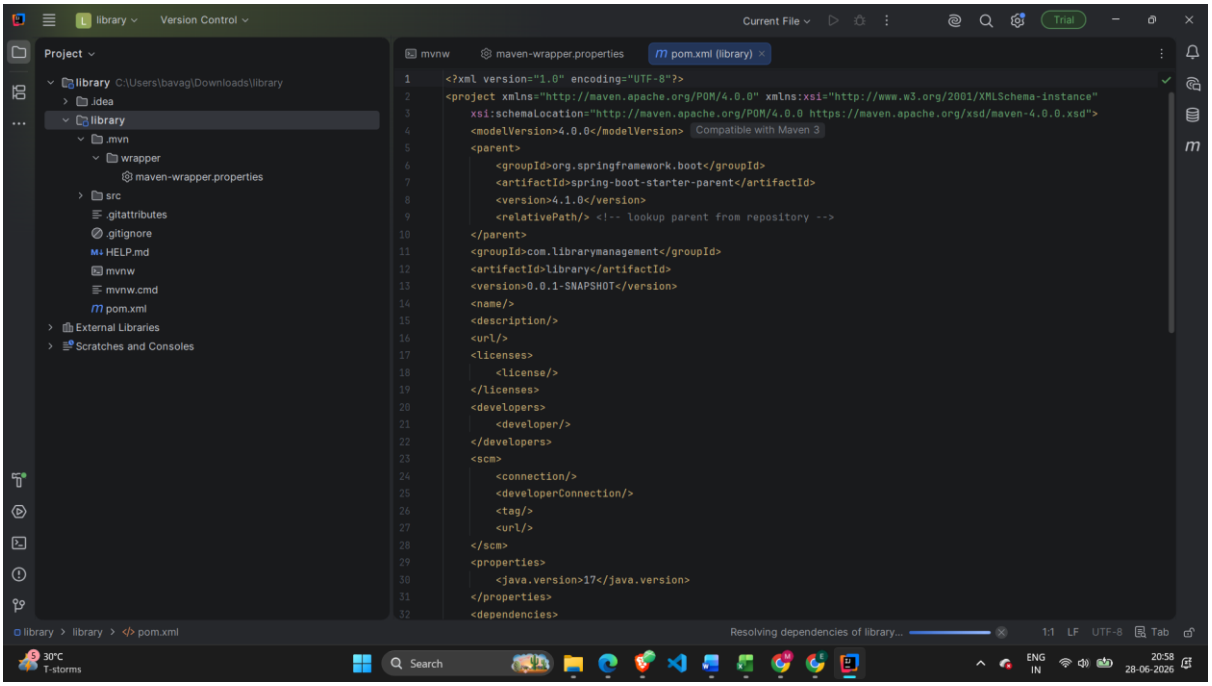
Steps:

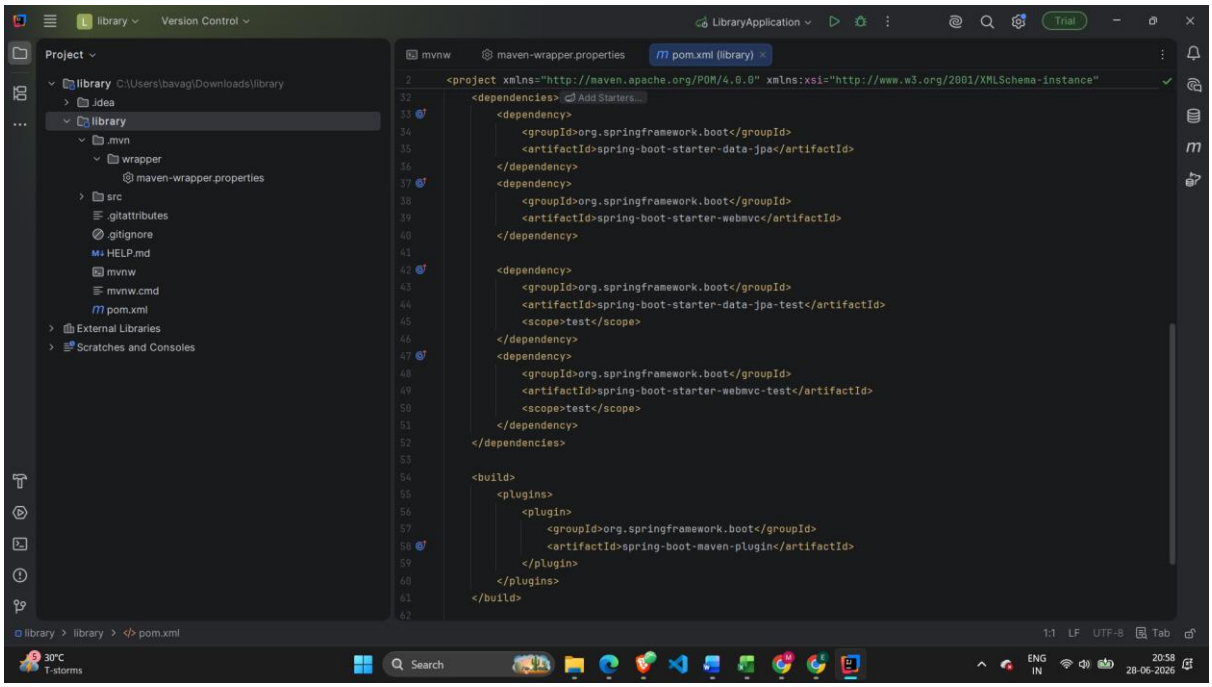
1. **Set Up a Spring Project:**
    - Create a Maven project named **LibraryManagement**.
    - Add Spring Core dependencies in the **pom.xml** file.
2. **Configure the Application Context:**
    - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
    - Define beans for **BookService** and **BookRepository** in the XML file.
3. **Define Service and Repository Classes:**
    - Create a package **com.library.service** and add a class **BookService**.
    - Create a package **com.library.repository** and add a class **BookRepository**.
4. **Run the Application:**
    - Create a main class to load the Spring context and test the configuration.

Created Maven Project

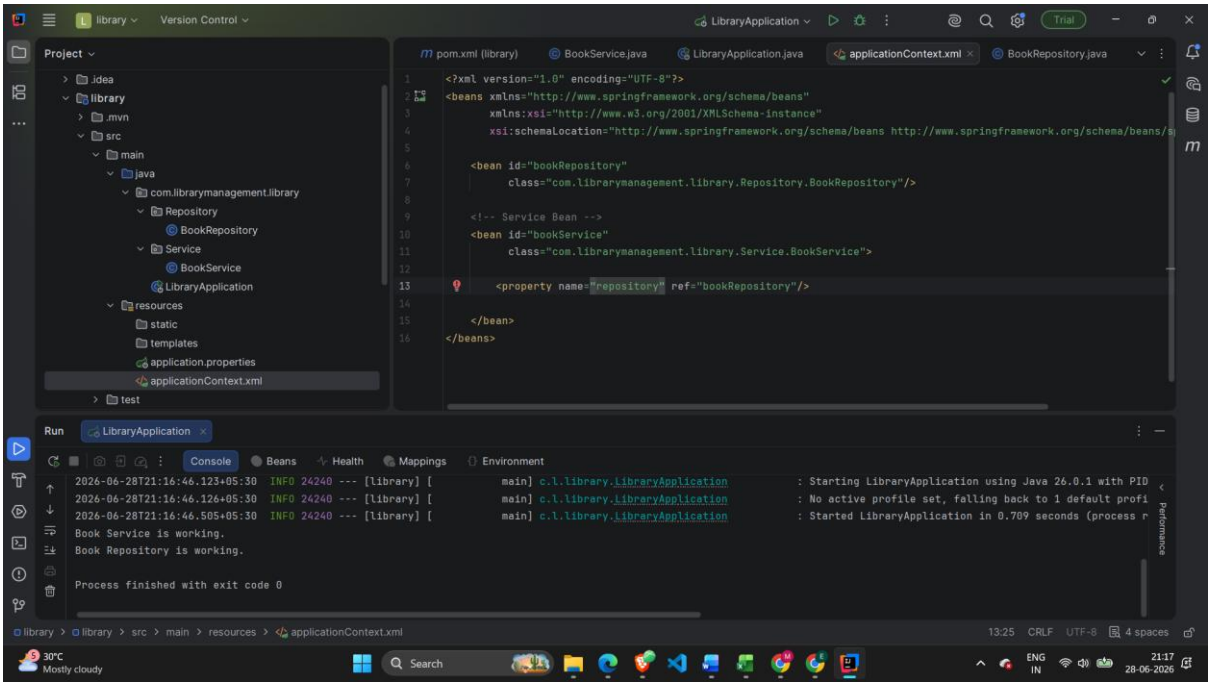


Pom.xml

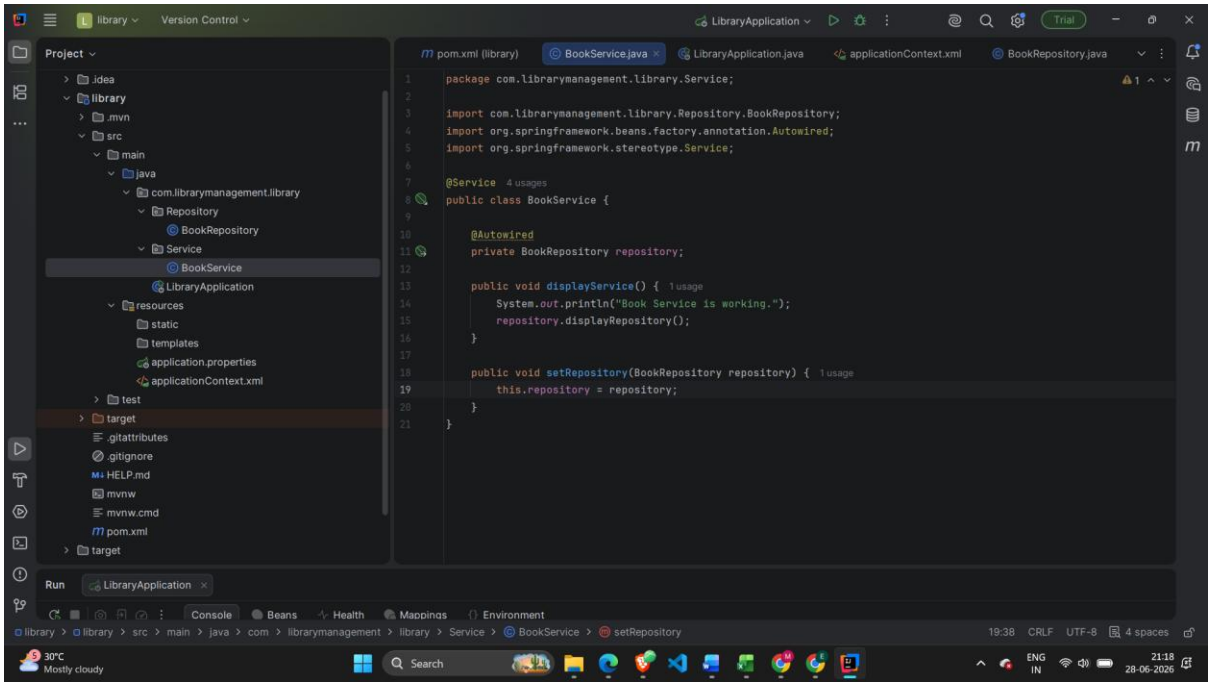




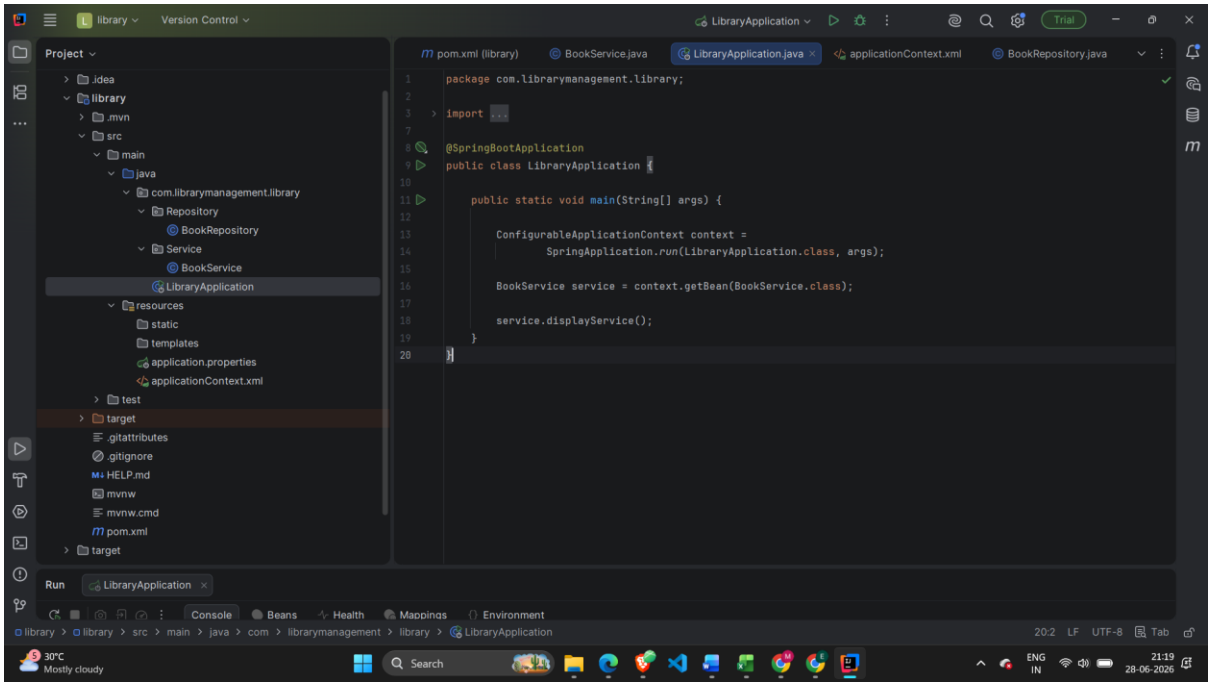
applicationContext.xml



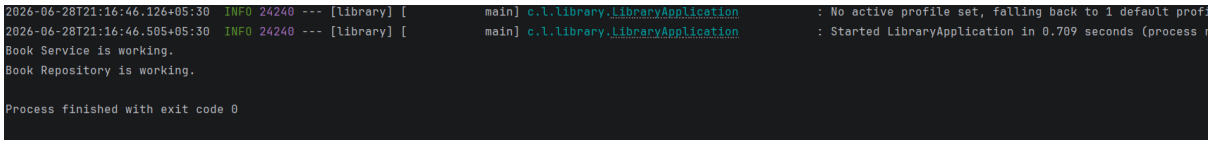
Service and repository classes



Main method



Output:



Exercise 2: Implementing Dependency Injection

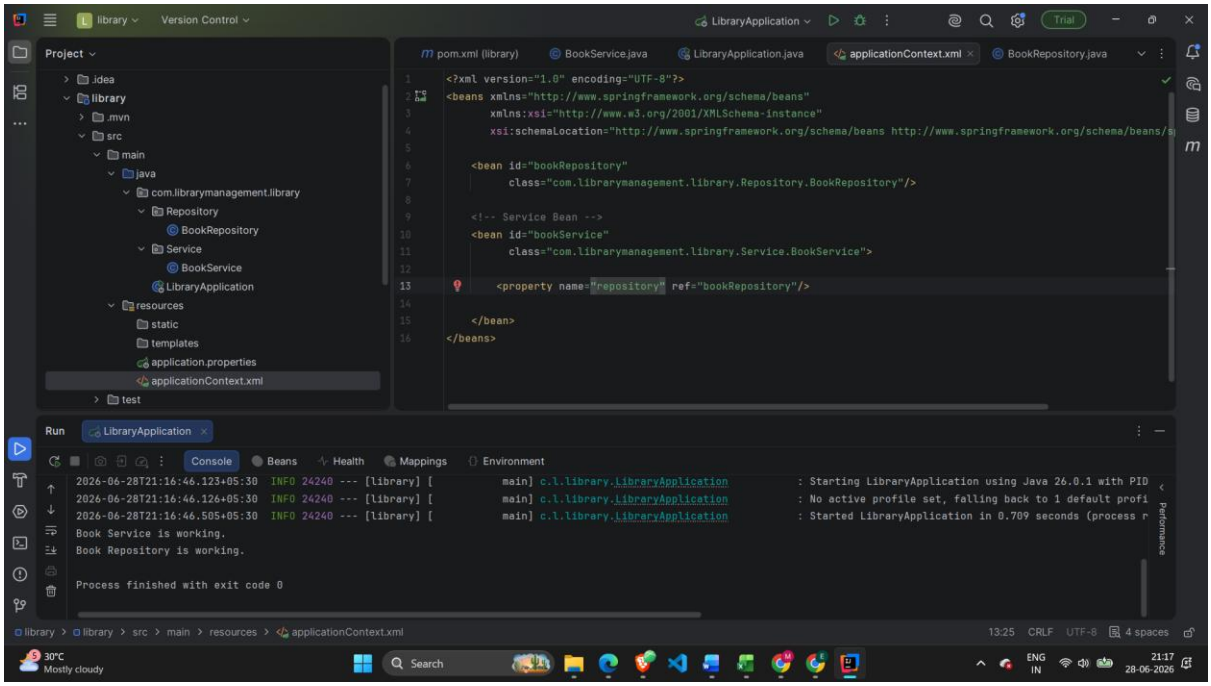
Scenario:

In the library management application, you need to manage the dependencies between the BookService and BookRepository classes using Spring's IoC and DI.

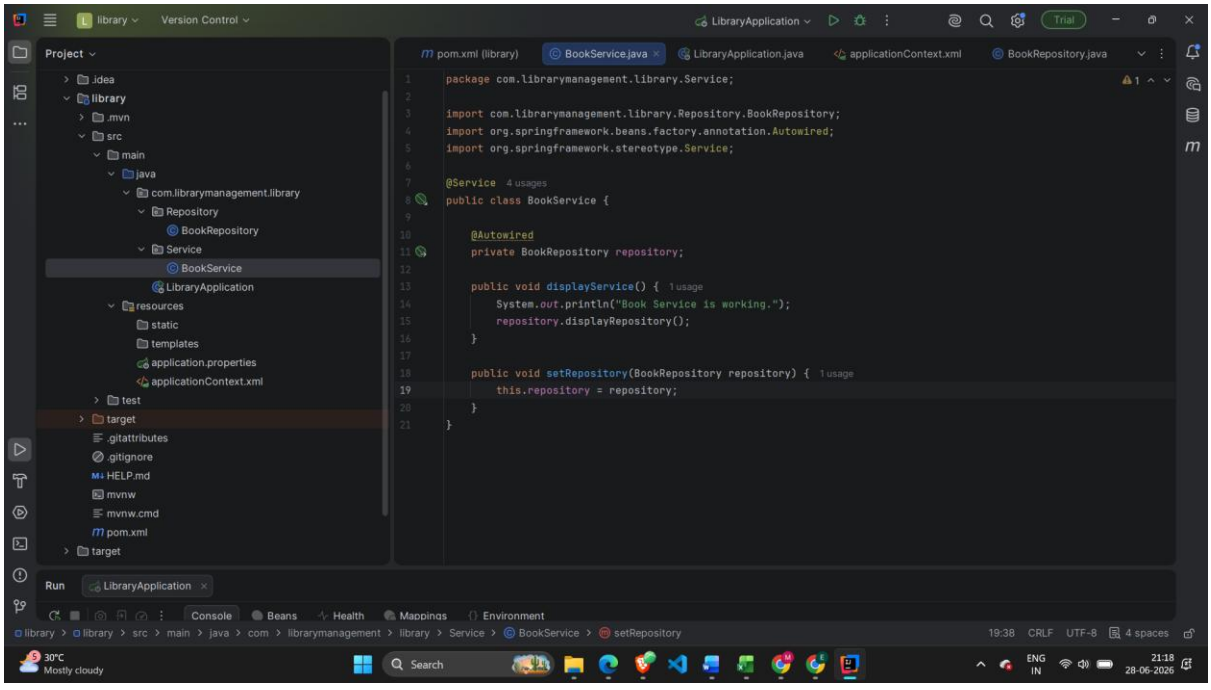
Steps:

1. **Modify the XML Configuration:**
    - Update **applicationContext.xml** to wire **BookRepository** into **BookService**.
2. **Update the BookService Class:**
    - Ensure that **BookService** class has a setter method for **BookRepository**.
3. **Test the Configuration:**
    - Run the **LibraryManagementApplication** main class to verify the dependency injection.

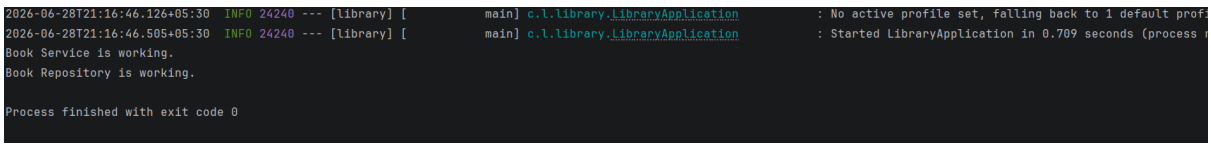
XML:



Update the BookService Class:



Test the Configuration:



Exercise 4: Creating and Configuring a Maven Project

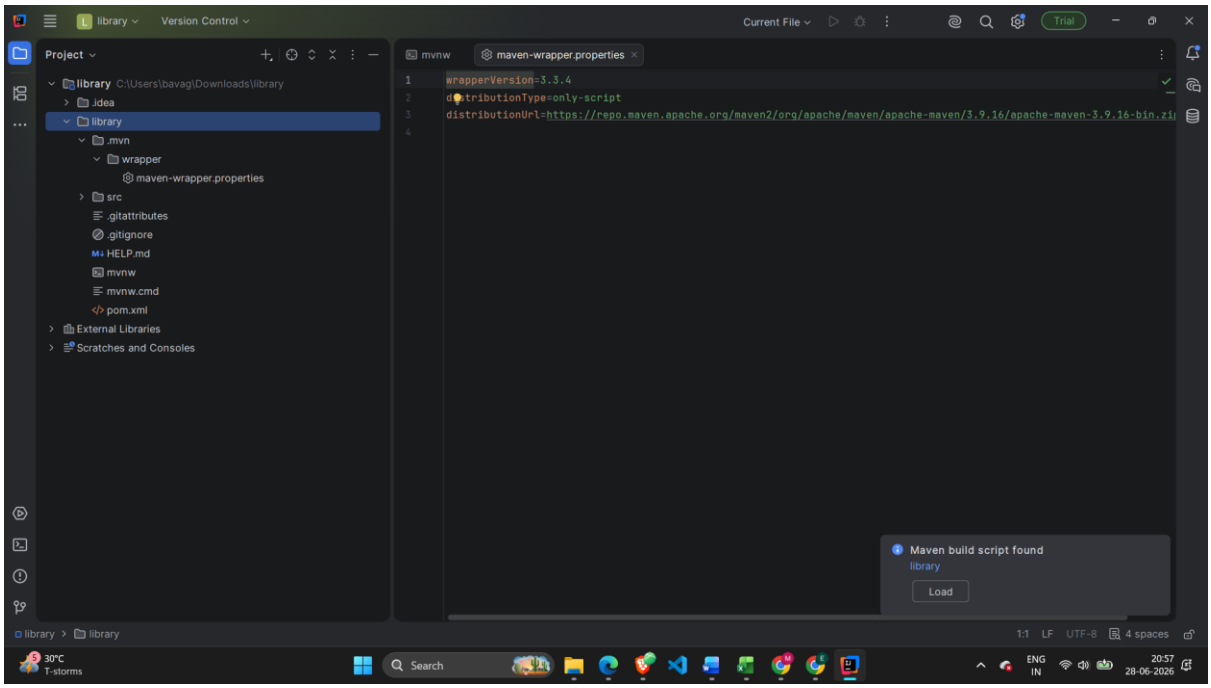
Scenario:

You need to set up a new Maven project for the library management application and add Spring dependencies.

Steps:

1. **Create a New Maven Project:**
    - Create a new Maven project named **LibraryManagement**.
2. **Add Spring Dependencies in pom.xml:**
    - Include dependencies for Spring Context, Spring AOP, and Spring WebMVC.
3. **Configure Maven Plugins:**
    - Configure the Maven Compiler Plugin for Java version 1.8 in the pom.xml file.

MAVEN project



Add Spring Dependencies in pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>

<groupId>com.librarymanagement</groupId>
<artifactId>LibraryManagement</artifactId>
<version>1.0-SNAPSHOT</version>

<properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
</properties>

<dependencies>

    <!-- Spring Context -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.30</version>
    </dependency>

    <!-- Spring AOP -->
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aop</artifactId>
        <version>5.3.30</version>
    </dependency>

    <!-- Spring Web MVC -->
```

```
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>5.3.30</version>
    </dependency>

  </dependencies>

  <build>
    <plugins>

      <!-- Maven Compiler Plugin -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.11.0</version>

        <configuration>
          <source>1.8</source>
          <target>1.8</target>
        </configuration>

      </plugin>

    </plugins>
  </build>

</project>
```

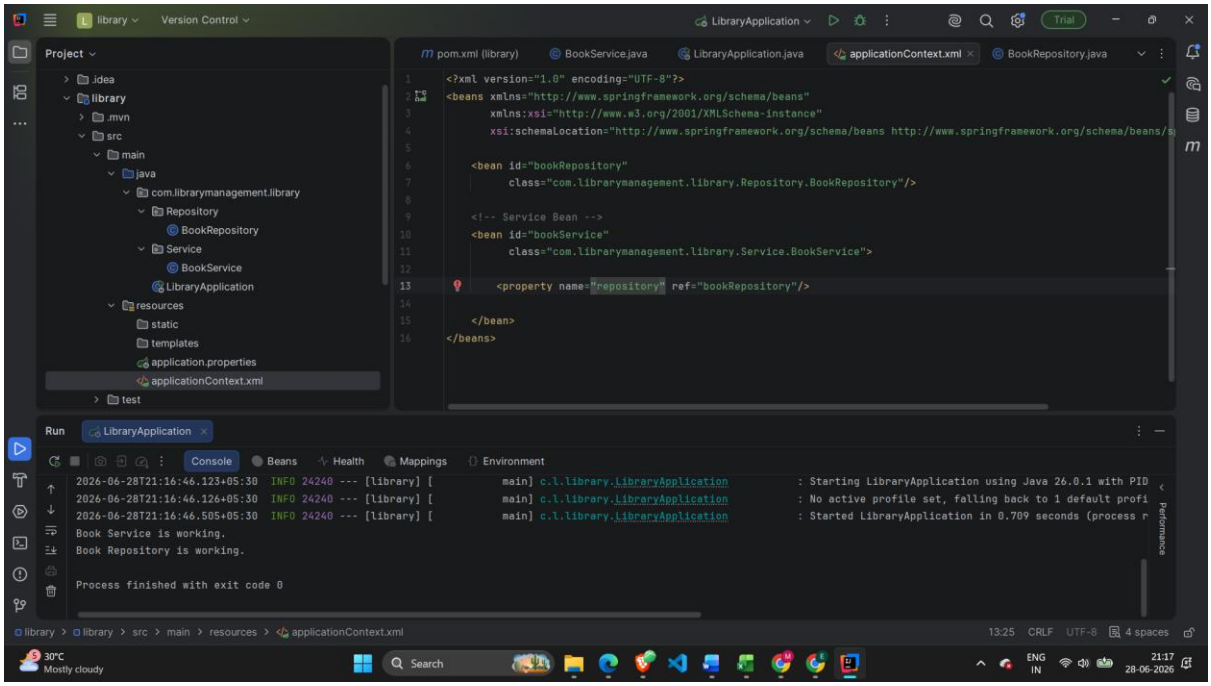
Exercise 5: Configuring the Spring IoC Container

Scenario:

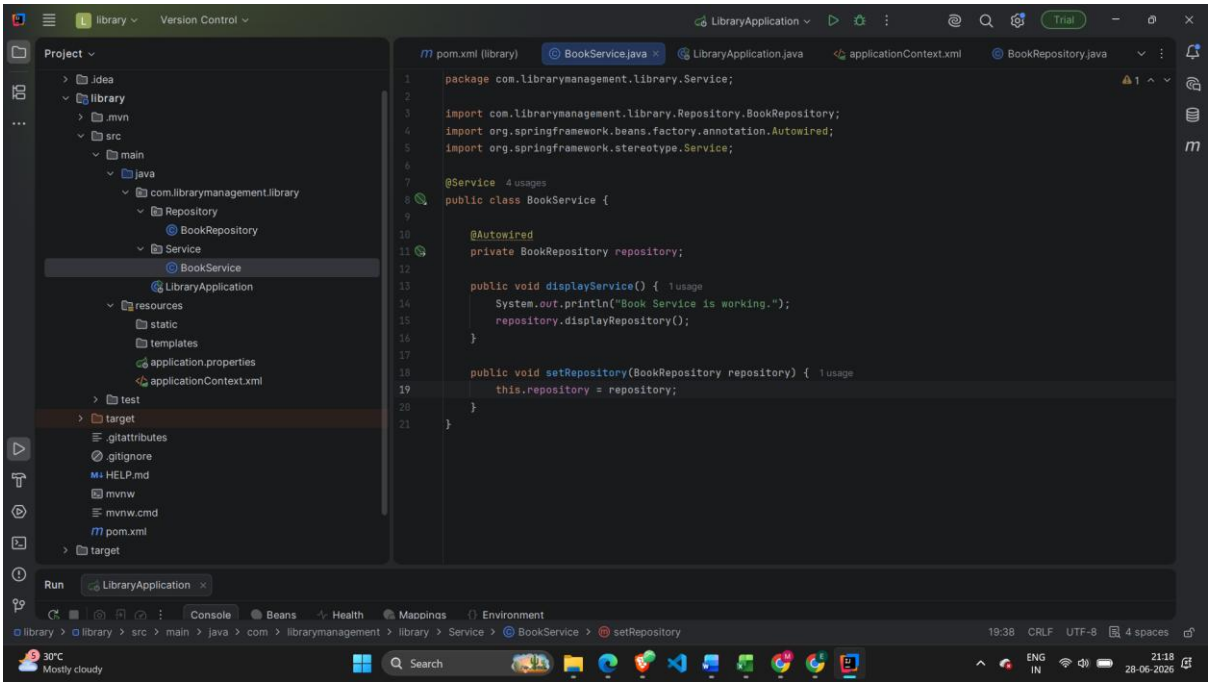
The library management application requires a central configuration for beans and dependencies.

Steps:

1. **Create Spring Configuration File:**
    - Create an XML configuration file named **applicationContext.xml** in the **src/main/resources** directory.
    - Define beans for **BookService** and **BookRepository** in the XML file.
2. **Update the BookService Class:**
    - Ensure that the **BookService** class has a setter method for **BookRepository**.
3. **Run the Application:**
    - Create a main class to load the Spring context and test the configuration.



Update the BookService Class:



Run the Application:

Exercise 7: Implementing Constructor and Setter Injection

Scenario:

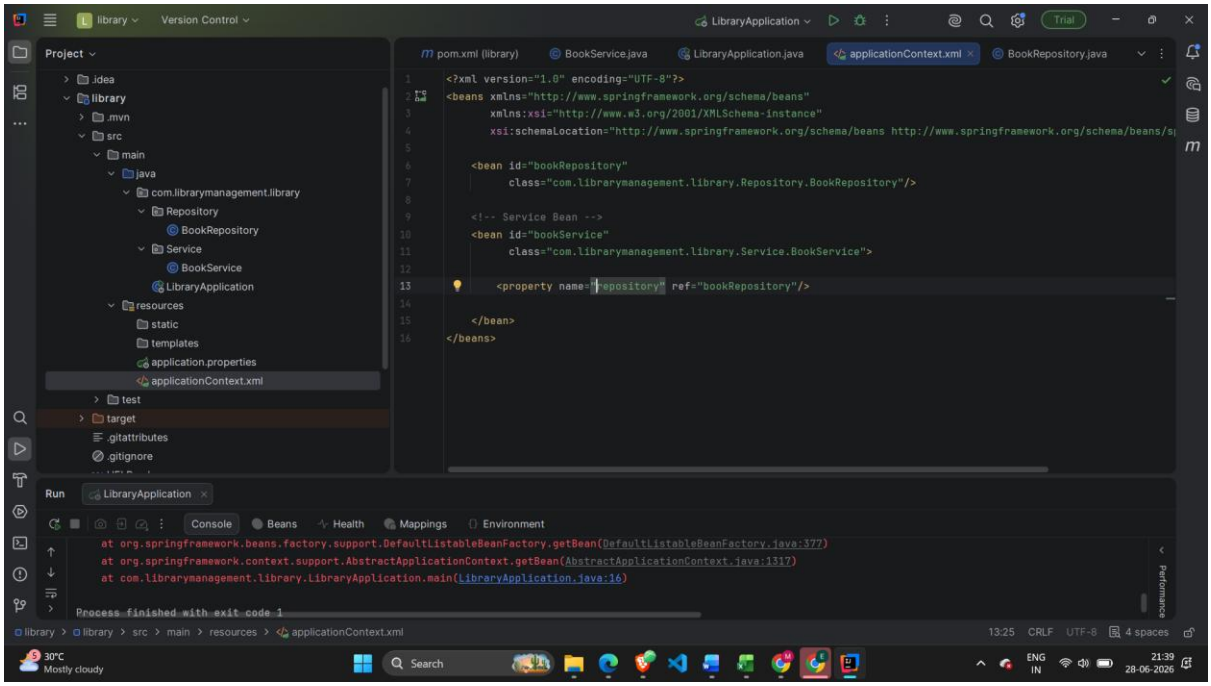
The library management application requires both constructor and setter injection for better control over bean initialization.

Steps:

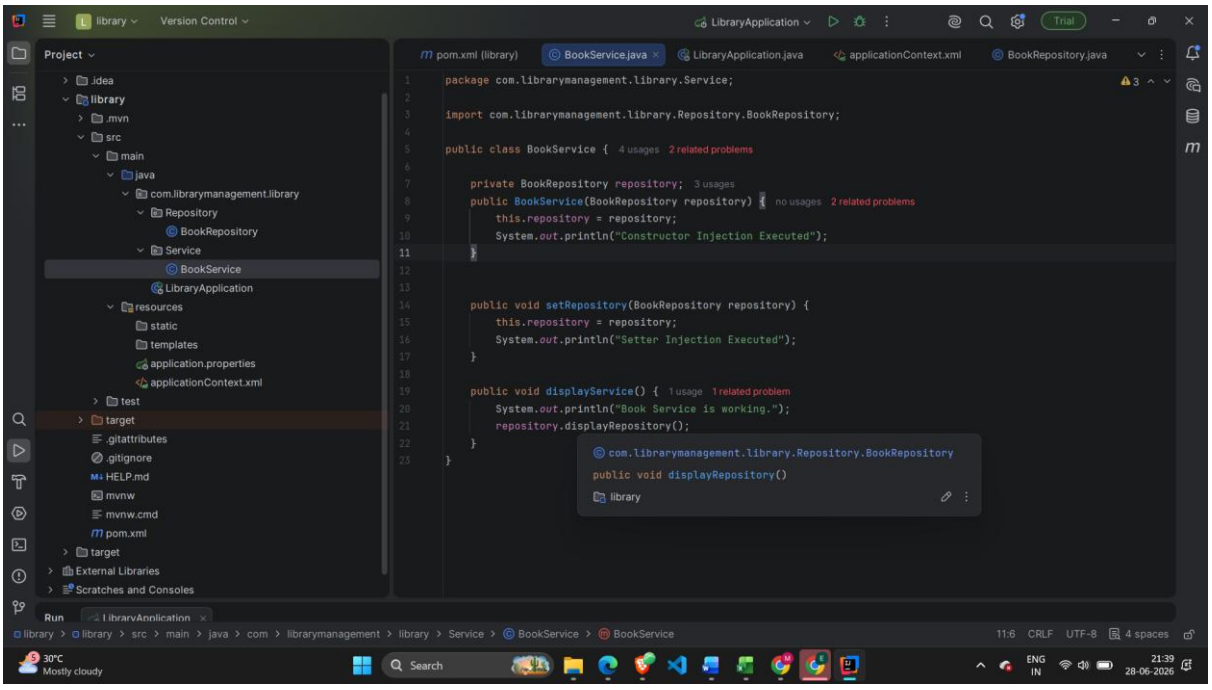
1. **Configure Constructor Injection:**
    - Update applicationContext.**xml** to configure constructor injection for **BookService**.
2. **Configure Setter Injection:**
    - Ensure that the **BookService** class has a setter method for **BookRepository** and configure it in **applicationContext.xml**.
3. **Test the Injection:**

- Run the **LibraryManagementApplication** main class to verify both constructor and setter injection.

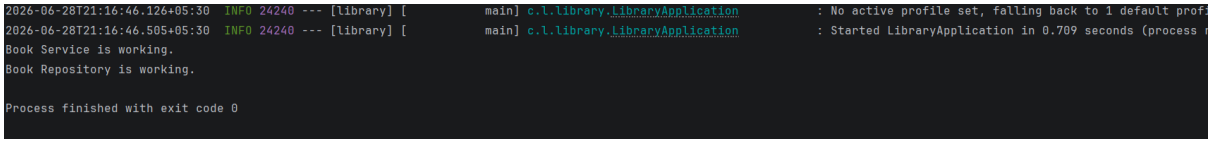
Configure Constructor Injection:



1. **Configure Setter Injection:**



Test the Injection:



Exercise 9: Creating a Spring Boot Application

Scenario:

You need to create a Spring Boot application for the library management system to simplify configuration and deployment.

Steps:

1. **Create a Spring Boot Project:**
    - Use **Spring Initializr** to create a new Spring Boot project named **LibraryManagement**.
2. **Add Dependencies:**
    - Include dependencies for **Spring Web, Spring Data JPA, and H2 Database**.
3. **Create Application Properties:**
    - Configure database connection properties in **application.properties**.
4. **Define Entities and Repositories:**
    - Create **Book** entity and **BookRepository** interface.
5. **Create a REST Controller:**
    - Create a **BookController** class to handle CRUD operations.
6. **Run the Application:**
    - Run the Spring Boot application and test the REST endpoints.

Adding dependencies

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.0</version>
```

```
  <relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.librarymanagement</groupId>
<artifactId>library</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name/>
<description/>
<url/>
<licenses>
  <license/>
</licenses>
<developers>
  <developer/>
</developers>
<scm>
  <connection/>
  <developerConnection/>
  <tag/>
  <url/>
</scm>
<properties>
  <java.version>17</java.version>
</properties>
<dependencies>

  <!-- Spring Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- Spring Data JPA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- H2 Database -->
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
```

```
        <scope>runtime</scope>
      </dependency>

    </dependencies>

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>

</project>
```

Create Application Properties:

```
# H2 Database Configuration
spring.datasource.url=jdbc:h2:mem:librarydb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# Enable H2 Console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

1. **Define Entities and Repositories:**

```
package com.librarymanagement.entity;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;

@Entity
public class Book {

    @Id
    private int id;
    private String title;
    private String author;

    public Book() {
    }

    public Book(int id, String title, String author) {
        this.id = id;
        this.title = title;
        this.author = author;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
```

```
    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }
}
```

BookRepository.java

```
package com.librarymanagement.repository;

import com.librarymanagement.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BookRepository extends JpaRepository<Book, Integer> {

}
```

BookController.java

```
package com.librarymanagement.controller;

import com.librarymanagement.entity.Book;
import com.librarymanagement.repository.BookRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/books")
public class BookController {

    @Autowired
    private BookRepository repository;

    // Get All Books
    @GetMapping
    public List<Book> getAllBooks() {
        return repository.findAll();
    }

    // Get Book By ID
    @GetMapping("/{id}")
    public Book getBook(@PathVariable int id) {
        return repository.findById(id).orElse(null);
    }
```

```
    // Add Book
    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return repository.save(book);
    }

    // Update Book
    @PutMapping("/{id}")
    public Book updateBook(@PathVariable int id,
                           @RequestBody Book book) {
        book.setId(id);
        return repository.save(book);
    }

    // Delete Book
    @DeleteMapping("/{id}")
    public String deleteBook(@PathVariable int id) {
        repository.deleteById(id);
        return "Book Deleted Successfully";
    }
}
```